VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

APPLICATIONS OF NATURAL LANGUAGE PROCESSING IN INDUSTRY

Project 2

FINE-TUNING INTENT DETECTION MODEL WITH BANKING DATASET

Lecturer: Dr. Nguyen Hong Buu Long

Ho Chi Minh City, 04/2026

Contents

1 Project requirements **2**

2 Task requirements **2**

2.1 Data preparation and processing 2

2.2 Fine-tuning the model with Unsloth 3

2.3 Inference implementation 3

2.4 Source code 4

2.5 Video demonstration 5

1 Project requirements

The objective of this lab is to study and apply fine-tuning techniques to a banking intent classification task using the **BANKING77** dataset and **Unsloth**. Specifically, students are required to complete the following tasks:

- Sample and construct a suitable subset of the BANKING77 dataset for banking intent classification.
- Perform data preprocessing and split the sampled dataset into training and testing sets.
- Fine-tune a text classification model using Unsloth and clearly describe all hyperparameters, techniques, and configurations used.
- Evaluate and compare the performance of the fine-tuned model on an independent test set.
- Implement a standalone inference file that loads the saved checkpoint and predicts the intent label of an input message.

2 Task requirements

2.1 Data preparation and processing

- Students must use the following dataset:
  - BANKING77: BANKING77.
- Students should sample and use only a subset of the dataset to ensure that training can be completed with available computational resources.
- Perform necessary preprocessing steps, including text normalization, label mapping, and basic cleaning if needed.
- Convert the selected intent labels into a format suitable for sequence classification.
- Split the sampled data into **train** and **test** sets. Students may optionally create a validation split from the training set for model selection.

2.2 Fine-tuning the model with Unsloth

- Students must refer to the official fine-tuning guide: Unsloth.
- Students may use platforms such as Google Colab, Kaggle, or a local machine.
- Clearly document all hyperparameters used:
  - Batch size.
  - Learning rate.
  - Optimizer.
  - Number of training steps or epochs.
  - Maximum sequence length.
  - Any regularization or augmentation techniques used.
- Save the model checkpoint after fine-tuning.

2.3 Inference implementation

- After training is completed, students must implement a standalone inference file.
- For consistency in grading, the inference interface must include exactly two main methods:
  - `__init__`: used to load the configuration file, tokenizer, and model checkpoint.
  - `__call__`: used to receive an input message and return the predicted label.
- The required format is shown below:

```python
class IntentClassification:
    def __init__(self, model_path):
        pass

    def __call__(self, message):
        ...
        return predicted_label
```

- The `model_path` must point to a configuration file that contains at least the path to the saved model checkpoint.

- The inference file must be able to load the saved checkpoint and predict the intent label for a single text input.
- Students should also provide a short usage example showing how the inference class is called after training.

2.4 Source code

Students are required to push all source code to their GitHub account with the minimum requirements following:

- Source code should be organized with a particular format, including folders and files (see example):

```
banking-intent-unsloth
   |-- scripts
   |     |-- train.py
   |     |-- inference.py
   |     |-- preprocess_data.py
   |
   |-- configs
   |     |-- train.yaml
   |     |-- inference.yaml
   |
   |-- sample_data
   |     |-- train.csv
   |     |-- test.csv
   |
   |-- train.sh
   |-- inference.sh
   |-- requirements.txt
   |-- README.md
```

- **README.md** must include all descriptions in order to set up environments, download, train, and run model.

2.5 Video demonstration

- Students must submit **one short video** demonstrating the inference result of the trained model.
- The video should clearly show:
  - How the inference script is executed.
  - At least one example input message.
  - The predicted intent label produced by the model
  - The final **accuracy** obtained on the test set.
- The video does not need to be long or heavily edited. A simple screen recording is sufficient.
- The recommended duration is **2–5 minutes**.
- Students should upload a video to Google Drive and attach a link to the README.md file (Students should ensure that this video is public).